
MEMORY SYSTEMS
Cache, DRAM, Disk
MORGAN KAUFMANN
BRUCE JACOB • SPENCER W. NG • DAVID T. WANG

INTEL 1020

# Memory Systems

## Cache, DRAM, Disk

| | |
|---|---|
| *Publisher* | Denise E.M. Penrose |
| *Acquisitions Editor* | Chuck Glaser |
| *Publishing Services Manager* | George Morrison |
| *Senior Production Editor* | Paul Gottehrer |
| *Developmental Editor* | Nate McFadden |
| *Assistant Editor* | Kimberlee Honjo |
| *Cover Design* | Joanne Blank |
| *Text Design* | Dennis Schaefer |
| *Composition* | diacriTech |
| *Interior printer* | Maple-Vail Book Manufacturing Group |
| *Cover printer* | Phoenix Color |

Morgan Kaufmann Publishers is an imprint of Elsevier.
30 Corporate Drive, Suite 400, Burlington, MA 01803, USA

This book is printed on acid-free paper.

© 2008 by Elsevier Inc. All rights reserved.

Designations used by companies to distinguish their products are often claimed as trademarks or registered trademarks. In all instances in which Morgan Kaufmann Publishers is aware of a claim, the product names appear in initial capital or all capital letters. Readers, however, should contact the appropriate companies for more complete information regarding trademarks and registration.

No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means—electronic, mechanical, photocopying, scanning, or otherwise—without prior written permission of the publisher.

Permissions may be sought directly from Elsevier's Science & Technology Rights Department in Oxford, UK: phone: (+44) 1865 843830, fax: (+44) 1865 853333, E-mail: permissions@elsevier.com. You may also complete your request online via the Elsevier homepage (http://elsevier.com), by selecting "Support & Contact" then "Copyright and Permission" and then "Obtaining Permissions."

**Library of Congress Cataloging-in-Publication Data**
Application submitted

ISBN: 978-0-12-379751-3

For information on all Morgan Kaufmann publications,
visit our Web site at *www.mkp.com* or *www.books.elsevier.com*

Working together to grow
libraries in developing countries

www.elsevier.com | www.bookaid.org | www.sabre.org

ELSEVIER BOOK AID International Sabre Foundation

OVERVIEW

# On Memory Systems and Their Design

Memory is essential to the operation of a computer system, and nothing is more important to the development of the modern memory system than the concept of the memory hierarchy. While a flat memory system built of a single technology is attractive for its simplicity, a well-implemented hierarchy allows a memory system to approach simultaneously the performance of the fastest component, the cost per bit of the cheapest component, and the energy consumption of the most energy-efficient component.

For years, the use of a memory hierarchy has been very convenient, in that it has simplified the process of designing memory systems. The use of a hierarchy allowed designers to treat system design as a modularized process—to treat the memory system as an abstraction and to optimize individual subsystems (caches, DRAMs [dynamic RAM], disks) in isolation.

However, we are finding that treating the hierarchy in this way—as a set of disparate subsystems that interact only through well-defined functional interfaces and that can be optimized in isolation—no longer suffices for the design of modern memory systems. One trend becoming apparent is that many of the underlying implementation issues are becoming significant. These include the physics of device and interconnect scaling, the choice of signaling protocols and topologies to ensure signal integrity, design parameters such as granularity of access and support for concurrency, and communication-related issues such as scheduling algorithms and queueing. These low-level details have begun to affect the higher level design process quite dramatically, whereas they were considered transparent only a design-generation ago. Cache architectures are appearing that play to the limitations imposed by interconnect physics in deep submicron processes; modern DRAM design is driven by circuit-level limitations that create system-level headaches; and modern disk performance is dominated by the on-board caching and scheduling policies. This is a non-trivial environment in which to attempt optimal design.

This trend will undoubtedly become more important as time goes on, and even now it has tremendous impact on design results. As hierarchies and their components grow more complex, *systemic* behaviors—those arising from the complex interaction of the memory system's parts—have begun to dominate. The real loss of performance is not seen in the CPU or caches or DRAM devices or disk assemblies themselves, but in the subtle interactions between these subsystems and in the manner in which these subsystems are connected. Consequently, it is becoming increasingly foolhardy to attempt system-level optimization by designing/optimizing each of the parts in isolation (which, unfortunately, is often the approach taken in modern computer design). No longer can a designer remain oblivious to issues "outside the scope" and focus solely on designing a subsystem. It has now become the case that a memory-systems designer, wishing to build a properly behaved memory hierarchy, must be intimately familiar with issues involved at all levels of an implementation, from cache to DRAM to disk. Thus, we wrote this book.

## Ov.1 Memory Systems

A memory hierarchy is designed to provide multiple functions that are seemingly mutually exclusive. We start at random-access memory (RAM): all microprocessors (and computer systems in general) expect a random-access memory out of which they operate. This is fundamental to the structure of modern software, built upon the von Neumann model in which code and data are essentially the same and reside in the same place (i.e., memory). All requests, whether for instructions or for data, go to this random-access memory. At any given moment, any particular datum in memory may be needed; there is no requirement that data reside next to the code that manipulates it, and there is no requirement that two instructions executed one after the other need to be adjacent in memory. Thus, the memory system must be able to handle randomly addressed[1] requests in a manner that favors no particular request. For instance, using a tape drive for this primary memory is unacceptable for performance reasons, though it might be acceptable in the Turing-machine sense.

Where does the mutually exclusive part come in? As we said, all microprocessors are built to expect a random-access memory out of which they can operate. Moreover, this memory must be *fast*, matching the machine's processing speed; otherwise, the machine will spend most of its time tapping its foot and staring at its watch. In addition, modern software is written to expect gigabytes of storage for data, and the modern consumer expects this storage to be cheap. How many memory technologies provide both tremendous speed and tremendous storage capacity at a low price? Modern processors execute instructions both out of order and speculatively—put simply, they execute instructions that, in some cases, are not meant to get executed—and system software is typically built to expect that certain changes to memory are permanent. How many memory technologies provide non-volatility and an *undo* operation?

While it might be elegant to provide all of these competing demands with a single technology (say, for example, a gigantic battery-backed SRAM [static RAM]), and though there is no engineering problem that cannot be solved (if ever in doubt about this, simply query a room full of engineers), the reality is that building a full memory system out of such a technology would be prohibitively expensive today.[2] The good news is that it is not necessary. Specialization and division of labor make possible all of these competing goals simultaneously. Modern memory systems often have a terabyte of storage on the desktop and provide instruction-fetch and data-access bandwidths of 128 GB/s or more. Nearly all of the storage in the system is non-volatile, and speculative execution on the part of the microprocessor is supported. All of this can be found in a memory system that has an average cost of roughly 1/100,000,000 pennies per bit of storage.

The reason all of this is possible is because of a phenomenon called *locality of reference* [Belady 1966, Denning 1970]. This is an observed behavior that computer applications tend to exhibit and that, when exploited properly, allows a small memory to serve in place of a larger one.

### Ov.1.1 Locality of Reference Breeds the Memory Hierarchy

We think linearly (in steps), and so we program the computer to solve problems by working in steps. The practical implications of this are that a computer's use of the memory system tends to be non-random and highly predictable. Thus is born the concept of *locality of reference*, so named because memory references tend to be localized in time and space:

- If you use something once, you are likely to use it again.
- If you use something once, you are likely to use its neighbor.

The first of these principles is called *temporal locality*; the second is called *spatial locality*. We will discuss them (and another type of locality) in more detail in *Part I: Cache* of this book, but for now it suffices to

[1] Though "random" addressing is the commonly used term, authors actually mean *arbitrarily* addressed requests because, in most memory systems, a *randomly* addressed sequence is one of the most efficiently handled events.

[2] Even Cray machines, which were famous for using SRAM as their main memory, today are built upon DRAM for their main memory.

say that one can exploit the locality principle and render a single-level memory system, which we just said was expensive, unnecessary. If a computer's use of the memory system, given a small time window, is both predictable and limited in spatial extent, then it stands to reason that a program does not need all of its data immediately accessible. A program would perform nearly as well if it had, for instance, a *two-level* store, in which the first level provides immediate access to a subset of the program's data, the second level holds the remainder of the data but is slower and therefore cheaper, and some appropriate heuristic is used to manage the movement of data back and forth between the levels, thereby ensuring that the most-needed data is usually in the first-level store.

This generalizes to the *memory hierarchy*: multiple levels of storage, each optimized for its assigned task. By choosing these levels wisely a designer can produce a system that has the best of all worlds: performance approaching that of the fastest component, cost per bit approaching that of the cheapest component, and energy consumption per access approaching that of the least power-hungry component.

The modern hierarchy is comprised of the following components, each performing a particular function or filling a functional niche within the system:

- **Cache (SRAM):** Cache provides access to program instructions and data that has very low latency (e.g., 1/4 nanosecond per access) and very high bandwidth (e.g., a 16-byte instruction block and a 16-byte data block per cycle => 32 bytes per 1/4 nanosecond, or 128 bytes per nanosecond, or 128 GB/s). It is also important to note that cache, on a per-access basis, also has relatively low energy requirements compared to other technologies.
- **DRAM:** DRAM provides a random-access storage that is relatively large, relatively fast, and relatively cheap. It is large and cheap compared to cache, and it is fast compared to disk. Its main strength is that it is just fast enough and just cheap enough to act as an operating store.
- **Disk:** Disk provides permanent storage at an ultra-low cost per bit. As mentioned, nearly all computer systems expect *some* data to be modifiable yet permanent, so the memory system must have, at some level, a permanent store. Disk's advantage is its very reasonable cost (currently less than 50¢ per gigabyte), which is low enough for users to buy enough of it to store thousands of songs, video clips, photos, and other memory hogs that users are wont to accumulate in their accounts (authors included).

Table Ov.1 lists some rough order-of-magnitude comparisons for access time and energy consumption per access.

Why is it not feasible to build a flat memory system out of these technologies? Cache is far too expensive to be used as permanent storage, and its cost to store a single album's worth of audio would exceed that of the

TABLE Ov.1 Cost-performance for various memory technologies

| Technology | Bytes per Access (typ.) | Latency per Access | Cost per Megabyte[a] | Energy per Access |
|---|---|---|---|---|
| On-chip Cache | 10 | 100 of picoseconds | $1–100 | 1 nJ |
| Off-chip Cache | 100 | Nanoseconds | $1–10 | 10–100 nJ |
| DRAM | 1000 (internally fetched) | 10–100 nanoseconds | $0.1 | 1–100 nJ (per device) |
| Disk | 1000 | Milliseconds | $0.001 | 100–1000 mJ |

[a]Cost of semiconductor memory is extremely variable, dependent much more on economic factors and sales volume than on manufacturing issues. In particular, on-chip caches (i.e., those integrated with a microprocessor core) can take up half of the die area, in which case their "cost" would be half of the selling price of that microprocessor. Depending on the market (e.g., embedded versus high end) and sales volume, microprocessor costs cover an enormous range of prices, from pennies per square millimeter to several dollars per square millimeter.

original music CD by several orders of magnitude. Disk is far too slow to be used as an operating store, and its average seek time for random accesses is measured in milliseconds. Of the three, DRAM is the closest to providing a flat memory system. DRAM is sufficiently fast enough that, without the support of a cache front-end, it can act as an operating store for many embedded systems, and with battery back-up it can be made to function as a permanent store. However, DRAM alone is not cheap enough to serve the needs of human users, who often want nearly a terabyte of permanent storage, and, even with random access times in the tens of nanoseconds, DRAM is not quite fast enough to serve as the only memory for modern general-purpose microprocessors, which would prefer a new block of instructions every fraction of a nanosecond.

So far, no technology has appeared that provides every desired characteristic: low cost, non-volatility, high bandwidth, low latency, etc. So instead we build a system in which each component is designed to offer one or more characteristics, and we manage the operation of the system so that the poorer characteristics of the various technologies are "hidden." For example, if most of the memory references made by the microprocessor are handled by the cache and/or DRAM subsystems, then the disk will be used only rarely, and, therefore, its extremely long latency will contribute very little to the average access time. If most of the data resides in the disk subsystem, and very little of it is needed at any given moment in time, then the cache and DRAM subsystems will not need much storage, and, therefore, their higher costs per bit will contribute very little to the average cost of the system. If done right, a memory system has an average cost approaching that of bottommost layer and an average access time and bandwidth approaching that of topmost layer.

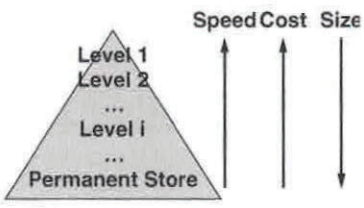



**FIGURE Ov.1:** A memory hierachy.

The memory hierarchy is usually pictured as a pyramid, as shown in Figure Ov.1. The higher levels in the hierarchy have better performance characteristics than the lower levels in the hierarchy; the higher levels have a higher cost per bit than the lower levels; and the system uses fewer bits of storage in the higher levels than found in the lower levels.

Though modern memory systems are comprised of SRAM, DRAM, and disk, these are simply technologies chosen to serve particular needs of the system, namely permanent store, operating store, and a fast store. Any technology set would suffice if it (a) provides permanent and operating stores and (b) satisfies the given computer system's performance, cost, and power requirements.

## Permanent Store

The system's permanent store is where everything lives ... meaning it is home to data that can be modified (potentially), but whose modifications must be remembered across invocations of the system (power-ups and power-downs). In general-purpose systems, this data typically includes the operating system's files, such as boot program, OS (operating system) executable, libraries, utilities, applications, etc., and the users' files, such as graphics, word-processing documents, spreadsheets, digital photographs, digital audio and video, email, etc. In embedded systems, this data typically includes the system's executable image and any installation-specific configuration information that it requires. Some embedded systems also maintain in permanent store the state of any partially completed transactions to withstand worst-case scenarios such as the system going down before the transaction is finished (e.g., financial transactions).

These all represent data that should not disappear when the machine shuts down, such as a user's saved email messages, the operating system's code and configuration information, and applications and their saved documents. Thus, the storage must be *non-volatile*, which in this context means not susceptible to power outages. Storage technologies chosen for permanent store include magnetic disk, flash memory, and even EEPROM (electrically erasable programmable read-only memory), of which flash memory is a special type. Other forms of programmable ROM (read-only memory) such as ROM, PROM (programmable ROM),

or EPROM (erasable programmable ROM) are suitable for non-writable permanent information such as the executable image of an embedded system or a general-purpose system's boot code and BIOS.[3] Numerous exotic non-volatile technologies are in development, including magnetic RAM (MRAM), FeRAM (ferroelectric RAM), and phase-change RAM (PCRAM).

In most systems, the cost per bit of this technology is a very important consideration. In general-purpose systems, this is the case because these systems tend to have an enormous amount of permanent storage. A desktop can easily have more than 500 GB of permanent store, and a departmental server can have one hundred times that amount. The enormous number of bits in these systems translates even modest cost-per-bit increases into significant dollar amounts. In embedded systems, the cost per bit is important because of the significant number of units shipped. Embedded systems are often consumer devices that are manufactured and sold in vast quantities, e.g., cell phones, digital cameras, MP3 players, programmable thermostats, and disk drives. Each embedded system might not require more than a handful of megabytes of storage, yet a tiny 1¢ increase in the cost per megabyte of memory can translate to a $100,000 increase in cost per million units manufactured.

## Operating (Random-Access) Store

As mentioned earlier, a typical microprocessor expects a new instruction or set of instructions on every clock cycle, and it can perform a data-read or data-write every clock cycle. Because the addresses of these instructions and data need not be sequential (or, in fact, related in any detectable way), the memory system must be able to handle *random access*—it must be able to provide instant access to any datum in the memory system.

The machine's operating store is the level of memory that provides random access at the microprocessor's data granularity. It is the storage level out of which the microprocessor could conceivably operate, i.e., it is the storage level that can provide random access to its storage, one data word at a time. This storage level is typically called "main memory." Disks cannot serve as main memory or operating store and cannot provide random access for two reasons: instant access is provided for only the data underneath the disk's head at any given moment, and the granularity of access is not what a typical processor requires. Disks are block-oriented devices, which means they read and write data only in large chunks; the typical granularity is 512 B. Processors, in contrast, typically operate at the granularity of 4 B or 8 B data words. To use a disk, a microprocessor must have additional buffering memory out of which it can read one instruction at a time and read or write one datum at a time. This buffering memory would become the *de facto* operating store of the system.

Flash memory and EEPROM (as well as the exotic non-volatile technologies mentioned earlier) are potentially viable as an operating store for systems that have small permanent-storage needs, and the non-volatility of these technologies provides them with a distinct advantage. However, not all are set up as an ideal operating store; for example, flash memory supports word-sized reads but supports only block-sized writes. If this type of issue can be handled in a manner that is transparent to the processor (e.g., in this case through additional data buffering), then the memory technology can still serve as a reasonable hybrid operating store.

Though the non-volatile technologies seem positioned perfectly to serve as operating store in all manner of devices and systems, DRAM is the most commonly used technology. Note that the only requirement of a memory system's operating store is that it provide random access with a small access granularity. Non-volatility is not a requirement, so long as it is provided by another level in the hierarchy. DRAM is a popular choice for operating store for several reasons: DRAM is faster than the various non-volatile technologies (in some cases *much* faster); DRAM supports an unlimited number of writes, whereas some non-volatile technologies start to fail after being erased and rewritten too many times (in some technologies, as few as 1–10,000 erase/write cycles); and DRAM processes are very similar to those used to build logic devices.

[3]BIOS = basic input/output system, the code that provides to software low-level access to much of the hardware.

CHAPTER 5

# Implementation Issues

This chapter presents the physical implementation (gate/transistor level) and architectural organization of modern-day caches, including a discussion of how this fits in a typical pipeline. One of the most important implementation issues today is that of dealing with the increasingly significant effects of technology scaling. In modern cache implementation, technology scaling exposes and/or exacerbates the following problems:

- The subthreshold leakage current caused by decreasing transistor threshold voltages becomes larger, increasing static power dissipation and requiring both evolutionary and revolutionary solutions to offset its effects.
- The decreasing amount of charge stored per bit makes it easier to corrupt a bit either by an external source (e.g., alpha emission, EMI) or by an internal source (e.g., crosstalk, ground bounce).
- Process variations increasingly cause mismatches of transistor characteristics in the implementation.
- The increasing contribution of wire propagation delay to the total delay must be taken into account in the pipeline's timing budget.

Some contemporary solutions are presented to counter some of the technology scaling problems present in deep and very deep submicron design.

## 5.1 Overview

To start things off, we discuss a sample cache operation, a cache read hit in moderate detail, exposing some of the implementation issues involved in its design.

Figure 5.1 shows an example cache organization: a two-way, set-associative cache with virtual addressing, along with a timing diagram showing the various events happening in the cache (to be discussed in much more detail in later sections).

Basically, the following steps involve:

1. Providing an address to the cache, along with an address strobe signal (ADS) confirming the validity of the address. A read/write signal (R/W#) is also sent to specify the operation.
2. The index part of the address chooses a word within the tag and data arrays of the two-way, set-associative cache (signified by the wordline signal, WL). This, in turn, causes the internal bitlines to develop a differential, which is amplified by sense amplifiers to produce a full-swing differential output voltage.
3. The translated address from the TLB is compared with the output of the tag array to decide if the cache access hit or miss. In case of a hit, the proper data is chosen among the two ways by controlling the output multiplexer and is forwarded. A cache miss requires the cache controller to perform

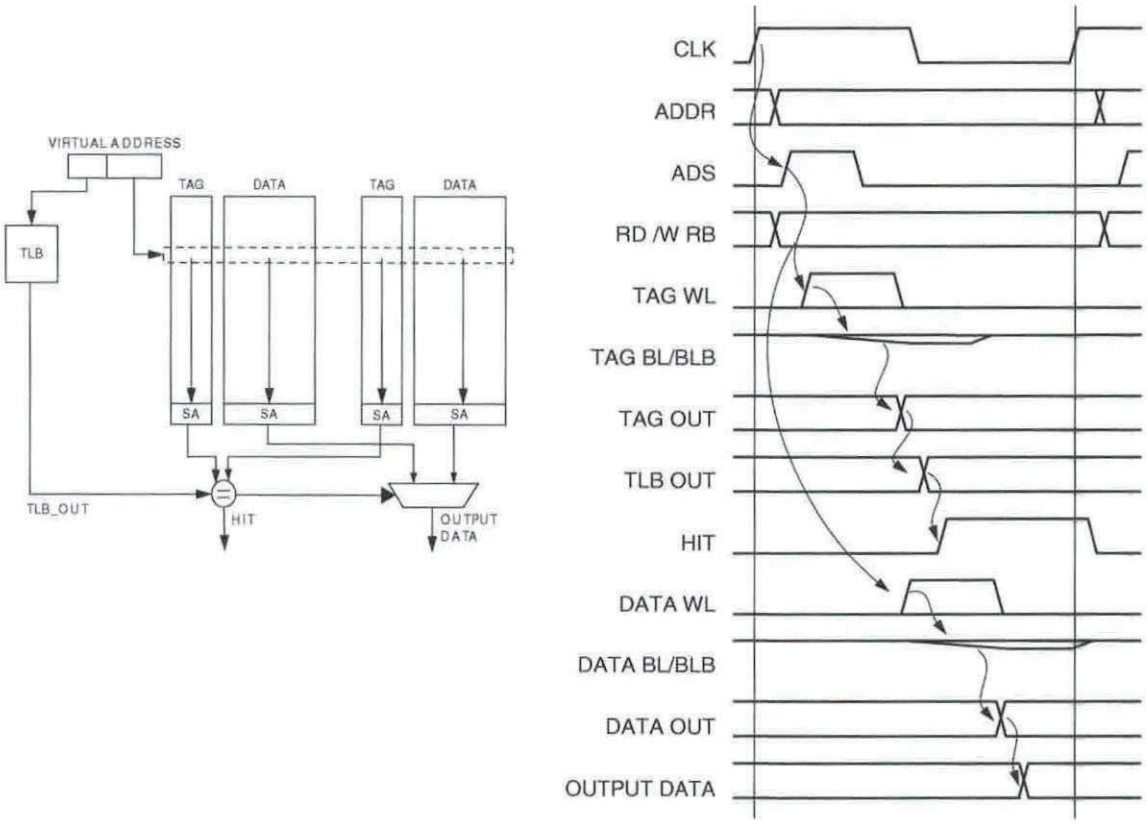



**FIGURE 5.1:** Block diagram of a two-way, set-associative cache organization, along with the timing diagram for a read operation.

a separate operation to retrieve data from external memory (or another level of cache) and to perform a write access to the cache.

We have now demonstrated the basic cache read operation and shown some of the blocks used in implementing a cache. We will proceed to more in-depth details, starting with the implementation of the basic storage structures comprising the tag and data array and moving on to how a cache is implemented and how these data arrays are used. Along the way, we also discuss advanced topics related to contemporary cache issues such as low-leakage operation.

## 5.2 SRAM Implementation

In this section, we discuss the implementation of a static random-access memory (SRAM). This is the type of memory used as the building block of most caches because of its superior performance over other memory structures, specifically DRAM. SRAM uses the same fabrication process as the microprocessor core, simplifying the integration of cache onto the processor die. By contrast, DRAM typically uses a different process that is sub-optimal for logic circuits, making the integration of logic and DRAM less attractive than the integration of logic and SRAM.

The discussion included here is complete enough to use as a basis for designing an SRAM from scratch.

### 5.2.1 Basic 1-Bit Memory Cell

This subsection describes how the most basic unit of an SRAM—a single data bit—is implemented. For SRAMs, the memory cell (MC) is implemented as two cross-coupled inverters accessed using two pass transistors, as shown in Figure 5.2.

This cross-coupled connection creates regenerative feedback that allows it to indefinitely store a single bit of data. This configuration shows a 1-bit memory cell with one R/W port that can be used for either a read or a write, but not both simultaneously. As shown in Figure 5.2, the bit is read by asserting a WL and detecting the voltage differential between the bitline pair, which is initially precharged to a logic high voltage.

The second phase, shown in Figure 5.1, is a write operation. The bitline is driven with a differential voltage from an external source to force the data onto the memory cell.

#### Physical Implementation of the 1-Bit Memory Cell

One of the assumptions of caches and memory, in general, is the ability to store digital data, and we have shown how a single bit can be stored by an SRAM using cross-coupled inverters.

This cross-coupled inverter has been implemented in different ways during its evolution, where the main difference has been how the inverter's pullup network is implemented. Figure 5.3 shows different circuit configurations that have been used to implement the memory cell.

Early SRAMs typically used either the full-CMOS or the polysilicon load memory cell configuration. The main advantage then of the poly-load configuration was its smaller area since only four transistors were required for one bit; the pullup was implemented using an additional highly resistive polysilicon layer such that the load was fabricated on top of the existing transistors, requiring less area.

With decrease in feature sizes, the amount of area occupied by the poly-load needed to produce a large enough resistance to overcome leakage currents that became too big. Along with its low soft error rate (SER) [List 1986], the poly-load implementation has now become impractical.

With the increase in space occupied by the poly-load, it started to be replaced by the poly-PMOS or thin-film-transistor (TFT) PMOS [Minato et al. 1987] load configurations, which could still be fabricated on top of the active NMOS, but with characteristics superior to the poly-load.

With the increasing on-chip integration of caches with microprocessor logic circuits, the additional process complexity required to implement the poly-PMOS/TFT-PMOS circuits on the same die as the digital circuits became too cumbersome.

An alternative to these circuits, the loadless four-transistor (LL4T) cell completely removes the pullup

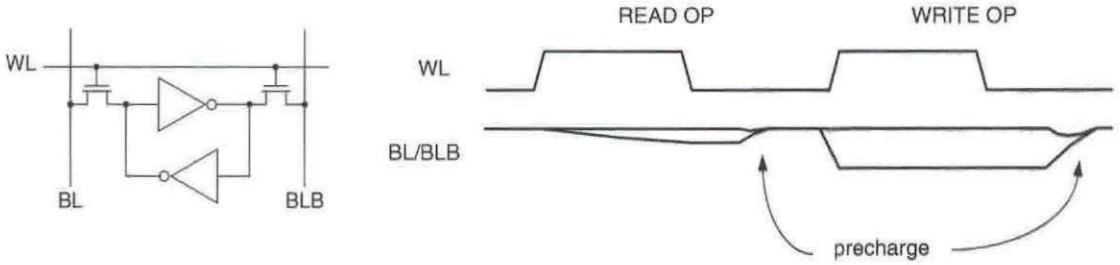



**FIGURE 5.2:** The basic SRAM cell showing cross-coupled inverters accessed through pass transistors. The state of important signals is also shown during a read and write access.

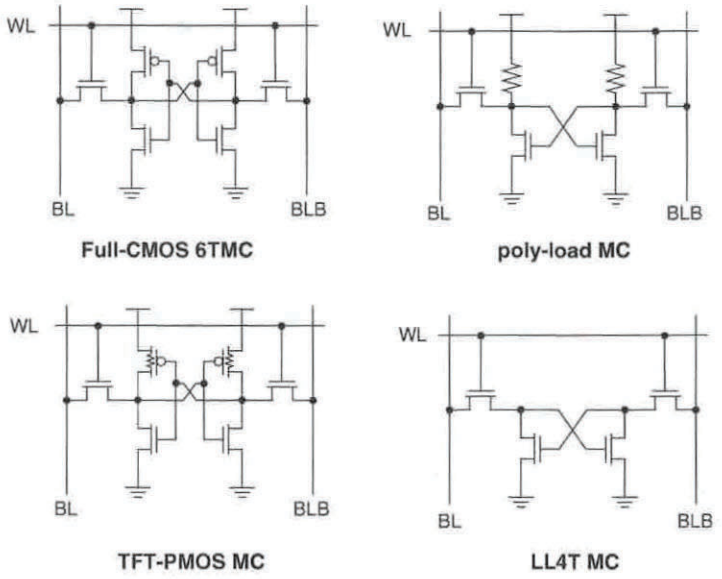



**FIGURE 5.3:** Different implementations of the SRAM cell.

load [Noda et al. 1998]. Although this scheme results in a smaller area compared to the full-CMOS implementation, because only four NMOS transistors are involved, the cell has to be very carefully designed to make sure that leakage currents do not interfere with the latching operation of the cell and its ability to retain data.

With continuous technology scaling, the charge stored within the cell decreases, while leakage current increases, making it more difficult to design reliable LL4T cells, especially if external radiation from alpha particles is taken into account.

Currently, most conventional designs use the full-CMOS six-transistor memory cell (6T MC), with different variations existing based on issues of sizing, physical layout, and transistor threshold voltages for low power (to be discussed later).

The rest of the discussion will be limited only to the 6T MC variant, and an example layout using MOSIS SCMOS rules for a 0.25-μm process is shown in Figure 5.4, along with its transistor-level circuit defining the access, driver, and pullup transistors.

### Transistor Sizing

The main initial considerations when sizing the transistors of the memory cell are the cell's area and its stability as measured by its static-noise margin (SNM). The SNM is defined as the amount of DC noise necessary to disturb the internal storage node of the cell and flip its stored data [Lohstroh et al. 1983]. The cell's stability affects the cell's SER and its sensitivity to PVT variations.

The cell's SNM can be calculated analytically by solving the cross-coupled inverter voltage transfer equations. Alternatively, it can be estimated using the graphical "maximum squares" method using the inverter's I/O voltage transfer function [Seevinck 1987], as shown in Figure 5.5.

stronger signal. Conversely, the track width for a lower performing head is narrower. Thus, higher bpi is coupled with lower tpi, and vice-versa. The product of the two is roughly constant, resulting in the same areal density. The number of tracks within each serpentine band of each surface will depend on the head. The same is true for the number of sectors per track. However, the total number of sectors within a band will be constant, or roughly constant, from surface to surface. For example, one surface may have 90 tracks in one band, with each track holding 1000 sectors. Another surface may have 100 tracks in that same band, but each track only holds 900 sectors. Yet, both surfaces have 90,000 sectors per band. This approach is more flexible, since it can have any mix of heads within the same drive, even all lower performing heads, and still be able to deliver the same total drive capacity.

Hence, adaptive formatting improves manufacturing efficiency. However, the user needs to realize that there may be variance in performance, especially data rate, from head to head and also from drive to drive.

## 25.6 Hybrid Disk Drive

In 2005, Microsoft submitted to the T13 technical committee for inclusion in the next ATA-8 standard a proposal for a new class of disk drives which incorporate a non-volatile cache inside the drive. To be more precise, the proposal was for a new set of commands for interfacing with such devices. This gives the host machine some control over how the non-volatile cache in the drive is to be used and managed. This new type of storage device is a hybrid disk drive because it will contain two different types of non-volatile memory: the magnetic media and the non-volatile cache.

### 25.6.1 Benefits

The expected benefits of a disk drive with a non-volatile cache are:

1. **Improved power management** When a drive has been spun down to save power during a period of inactivity, any new write command received from the host can be written to the non-volatile cache without having to spin up the drive. Such writes are accumulated either until the non-volatile cache is full or when the drive needs to be spun up for other reasons, such as to do a cache-miss read. Since spinning up the drive consumes a considerable amount of power, this is especially beneficial in a mobile system which runs on battery power.
2. **Improved drive reliability** Spinning up and down the drive can add to its wear and tear. By reducing the number of such cycles, the reliability of the drive can be improved. Also, the shock resistance of a drive is much higher when it is spun down and its heads are safely unloaded. In reducing the amount of time the drive is spinning, its exposure to shock damage is reduced.
3. **Faster boot and application loading times** By saving the boot image and the state of the system into the non-volatile cache when a machine is either shut down or put into hibernation, the next machine startup can load from the non-volatile cache without having to wait for the drive to be fully spun up and ready to read, which can take seconds. Similarly, applications that are stored in the nonvolatile memory can be loaded into the system more quickly.

### 25.6.2 Architecture

The new proposal does not specify the architecture for the new hybrid disk drive, so different designs and implementations are possible. Neither does the proposal specify what technology is to be used for the non-volatile cache. At this point in time, flash memory is the most likely candidate, in part due to its relatively low cost. In the future, other technologies such as MagRAM or MEMS memory [Carley et al. 2000, Albrecht et al. 2004] may be better choices.

The high-level architecture for a flash-based hybrid *Hard Disk Drive* (HDD) will likely look something like that shown in Figure 25.11. The flash memory will be placed on the internal data bus alongside the DRAM buffer/cache. There are several shortcomings with flash memory that the designer needs to be aware of and take into consideration while architecting a hybrid drive.

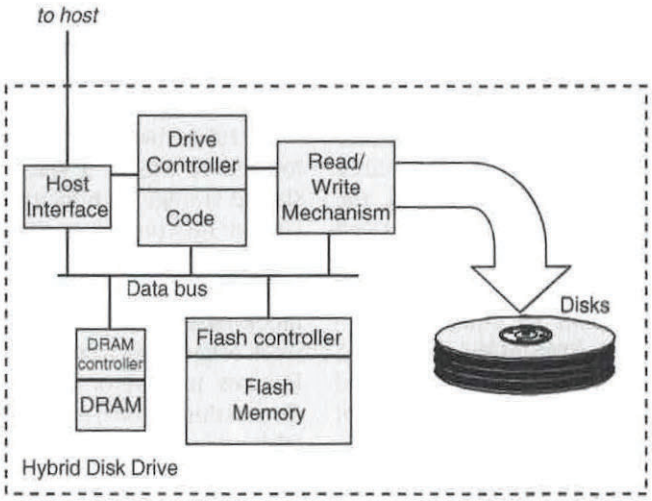



**FIGURE 25.11:** High-level architecture of a hybrid disk drive.

1. **Page write** Writing to flash memory generally has to be done in pages. The page size depends on the flash technology used and the manufacturer.
2. **Erase before write** A block, consisting of multiple pages, must first be erased before it can be written. Erasure time is very slow.
3. **Limited write cycles** Current flash technology has a limited number of cycles that each bit can be written, typically around 100,000 times. There are two general approaches for handling this limit. The first one is wear-monitoring, in which the number of write cycles for each block of flash memory is monitored. A block that has reached its limit will be permanently mapped out and made unavailable for further use. A second approach is wear-leveling, in which all the blocks in the flash memory are managed such that they all get used evenly. This usually means using the flash memory in a circular fashion.
4. **Low data rate** Flash write data rate is about an order of magnitude lower than that of a magnetic disk, read data rate is about one half, and erasure takes milliseconds. Thus, performance is lower compared to disk access when transferring a large amount of sequential data.

### 25.6.3 Proposed Interface

The non-volatile memory is used as a cache to the drive, rather than an extension to the drive's address space. The main salient feature of the proposed interface is the definition of a *non-volatile cache pinned set*. The host system can specify to the hybrid drive a set of LBAs that are to be pinned in the cache. An "Add LBAs to pinned set" command is used to add a new range of LBAs to the set, and a companion "Remove LBAs from pinned set" command is to unpin a range of LBAs. Spaces left over in the non-volatile memory are free to be used by the drive for write caching or other internal uses.

An interesting feature of the "Add LBAs to pinned set" command is the "Populate Immediate" flag. If this flag is set in the command, the drive is to prefetch the content of the LBA range from the disk. This usage is envisioned for the pinned set to be used for preloading boot files into the non-volatile cache in preparation for a shutdown and the next reboot. If

the flag is not set, then the drive will take no action to fetch data from the disk into those locations. Rather, it is anticipated that the host will send down write data to those LBAs. An example of this usage model is the writing of the hibernation file in preparation for the host system going into hibernation. After either a cold reboot or a wakeup from hibernation, the previously pinned LBAs can be unpinned and freed up for write caching use.

The proposed interface for nonvolatile cache can be found in the draft document "AT Attachment 8—ATA/, ATAPI Command Set (ATA8-ACS)" of the T13 Standard Committee (see www.T13.org). It is expected that the proposal will officially be adopted as part of ATA standard by the end of 2007. Microsoft's new Vista operating system supports hybrid HDDs, although at this time no hybrid HDDs are generally available from any disk manufacturer for evaluation yet.

## 25.7 Object-Based Storage

A disk drive, like all other digital storage devices, understands that it is being asked to store and retrieve bits of 0's and 1's. A disk drive is not cognizant of the meaning of the data that it is handling. With today's block interface, a drive is asked to read or write blocks of data, with each block being some multiple of its physical sector size. It does not know, or care, whether the block is part of a directory, part of a file, or a complete file. While this paradigm makes the storage device easier to implement and keeps the interface simple, from a system point of view it is not the most efficient.

A more efficient disk architecture would be for the disk drive itself to be cognizant of the space allocation aspect of the file system being used to manage its storage space. With such knowledge, it can perform lookahead prefetch much more intelligently and accurately. This would improve the effectiveness of the disk drive's cache. Furthermore, a user can access his data in the disk more directly without entailing all the file system directory lookup and update traffic going back and forth between the host and the drive. These and some other benefits can be derived from the proposed *object-based storage device* (OSD[3]) architecture [Mesnier et al. 2003].

In 1999, the OSD Technical Working Group was formed to create a self-managed, heterogeneous, shared storage architecture by moving low-level file storage functions into the storage device itself. The storage device would be accessed using a new object-based standard interface instead of the traditional block-based interfaces of SCSI or ATA. Building on work originating from the Network Attached Storage Devices project of the National Storage Industry Consortium (NSIC) and the Network Attached Secure Disk project at Carnegie Mellon University, the group developed requirements, standard definitions, and prototype demonstrations for OSD storage subsystems. The working group was later adopted by the Storage Networking Industry Association (SNIA), and collaboration with the ANSI T10 technical committee was established in 2000. Since mid-2004, OSD has been a standard extension to SCSI approved by T10.

Because it is defined as a new command set, it can be used on standard SCSI physical interfaces. This also means OSD architecture can be implemented either natively in a disk drive or in some higher level storage controller. So far, OSD exists mostly on paper only. For OSD to be generally adopted and embraced by the storage industry, it will require a massive amount of operating system changes, particularly file systems and databases. New storage devices with the OSD interface implemented will be needed too, naturally. Major software houses are not likely to produce OSD-enabled software until OSD hardware is generally available, while major storage vendors are not likely to manufacture such devices until software is available to use them. Some catalytic event is likely to be needed for OSD to break out of this catch-22 mode.

### 25.7.1 Object Storage Main Concept

The main idea of OSD is fairly simple. Each piece of user data, e.g., a file or a database record, and its

[3]Originally, OBSD.

Computer Systems Design / Computer Hardware

# MEMORY SYSTEMS
## Cache, DRAM, Disk

**Bruce Jacob**, *University of Maryland at College Park*
**Spencer W. Ng**, *Hitachi Global Storage Technologies*
**David T. Wang**, *MetaRAM*

"*Memory Systems: Cache, DRAM, Disk* fills a huge void in the literature about modern computer architecture...Jacob, Ng, and Wang have created one of the truly great technology books that turns reading about bits and bytes into an exciting journey towards understanding technology."
— Michael Schuette, *VP of Technology Development at OCZ Technology*

"*Memory Systems: Cache, DRAM, Disk* is remarkable in both its scope and depth.... This book will doubtless be the definitive reference for students and designers of memory systems for many years to come."
— Jim Smith, *University of Wisconsin – Madison*

The performance gap between processor and memory speeds has increased dramatically. System designers have talked for over a decade about "hitting the memory wall," a condition in which overall system speed is significantly compromised by the memory system's inability to keep pace with the processor. That condition is today's reality—the memory system has become the most critical performance bottleneck, and memory subsystems design is now the greatest challenge facing design engineers.

*Memory Systems: Cache, DRAM, Disk* is the first book to cover comprehensively the logical design and operation, physical design and operation, performance characteristics and resulting design trade-offs, and the energy consumption of modern memory hierarchies. This book describes the physical design and detailed software emulation of the entire memory hierarchy, from cache to disk and prepares designers to tackle the challenging optimization problems that result from the side effects that can appear at any point in the entire memory hierarchy.

### CONTENT HIGHLIGHTS

- Provides a comprehensive, in-depth understanding of all levels of the system hierarchy—cache, DRAM, and disk.
- Describes a holistic approach for evaluating the system-level effects of all design choices.
- Includes a link to the open-source software DRAMsim, a detailed and accurate parameter-driven simulator of modern DRAM-based memory systems that models performance and energy consumption for each component.

### ABOUT THE AUTHORS

**Bruce Jacob** is an Associate Professor of Electrical and Computer Engineering at the University of Maryland at College Park.

**Spencer W. Ng** is a senior technical staff member with Hitachi Global Storage Technologies. He has been a researcher in the field of storage for over twenty years, and is the holder of about twenty issued U.S. patents.

**David T. Wang** is a senior technical staff member at MetaRAM, a memory systems startup in Silicon Valley.

Online support materials for this book are available at
*textbooks.elsevier.com/9780123797513*

Cover photograph: istockphotos.com® Mypokcik

ELSEVIER
MORGAN KAUFMANN PUBLISHERS
AN IMPRINT OF ELSEVIER SCIENCE
www.mkp.com

ISBN 978-0-123-79751-3
9 780123 797513

JACOB
NG
WANG



# MEMORY SYSTEMS

## Cache, DRAM, Disk

MK
MORGAN
KAUFMANN